AI in Transportation: machines that learn by reward

Three jobs, one recipe: try something, get points, update one score. Repeat thousands of times.

Case 1 · Warehouse robot

Learns a route to the packing station from points alone.



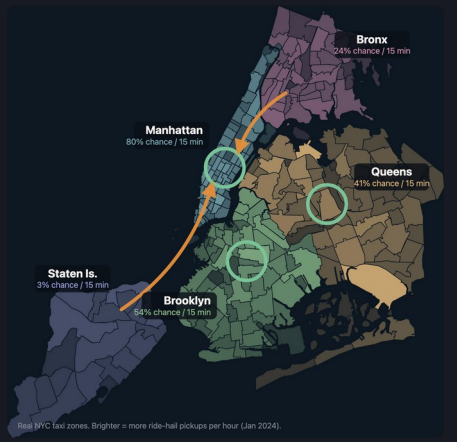
Case 2 · Traffic light

Learns when to switch, with safety rules it cannot break.



Case 3 · Ride-hail driver

Learns where to wait in New York, from real 2024 trip data.



ITAI 2372 | Houston Community College | Prof. Yuexin Mao

PLAN FOR THE SESSION

Tonight: one idea, three cases, one safety question

SEGMENT	WHAT STUDENTS LEARN	OPEN
Opening	The industry map, smart driving, learning by reward	
Case 1 · Warehouse robot	Points, the score table, the update, test drive, when the world changes	M7_Robot_Training_Lab.html
Case 2 · Traffic light	No finish line, safety rules, fair baselines, what you measure	M7_Traffic_Signal_Lab.html
Break		
Case 3 · Ride-hail driver	Real NYC data, delayed reward, who gets served	M7_Ride_Hail_Lab.html
Close	Safety and accountability, check yourself, labs and notebooks	M7_Notebook_*.html

Every lesson page runs offline in a browser. Every notebook reproduces the page's numbers exactly.

AI across transportation: six places it already works

Transportation runs on decisions that repeat all day. The three tagged areas are tonight's cases.

CASE 2

Roads & traffic

Signals that adjust to the cars actually waiting; cameras that spot crashes; apps that predict travel time.

COMING UP

Smart driving

Assistance that brakes or steers for you; driverless robotaxis inside mapped service areas.

Freight & delivery

Route plans for thousands of vans every morning; delivery-time estimates; truck load planning.

CASE 1

Warehouses

Robot fleets carry shelves to workers; software plans every robot's path so they do not collide.

CASE 3

Ride-hail & transit

Matching riders to nearby drivers; surge prices; where idle cars should wait; bus demand forecasts.

Rail, air & sea

Predicting train and aircraft part failures; crew and gate schedules; crane plans at container ports.

Every system runs the same loop: sense, predict, decide, act

Decisions repeat, and each one changes what happens next. That is the kind of problem learning by reward is built for.

Sense

Collect signals about the world.

- Cameras and lidar on vehicles
- Detectors in the road at intersections
- GPS pings from phones and fleets
- Tags on warehouse floors

Seen in Module 5 (sensors)

→

Predict

Estimate what happens next.

- Travel and arrival times
- Ride demand by area and hour
- Equipment that needs service
- What an object in an image is

Seen in Modules 3, 5 and 6

→

Decide

Choose an action, again and again.

- Robot routes in a warehouse (Case 1)
- When a light switches (Case 2)
- Where a ride-hail car waits (Case 3)
- Routing trucks, trains and buses

Tonight: learning by reward

→

Act

Carry it out, with people accountable.

- Drivers, operators, dispatchers
- Engineers who set the safety rules
- Companies that answer for the system
- Regulators and the public

Closing: who answers

AI at work today: four real deployments

Numbers are the operators' own public claims. Notice how many systems are optimization and prediction, not only learning by reward.

AMAZON ROBOTICS

1 million

robots deployed in its operations

An AI model, DeepFleet, coordinates how the fleet moves: 10% less robot travel time.

Fleet coordination (Case 1)

Source: Amazon, June 2025

SURTRAC, PITTSBURGH

–25%

travel time through the signals

Each intersection re-plans its green times from the cars it detects and tells its neighbors. Waiting fell 40%.

Signal scheduling (Case 2)

Source: Carnegie Mellon (CMU)

UPS ORION

10–14 mi

fewer miles per driver per day

AI and machine learning map out the best order and route for each driver's stops.

Route optimization

Source: UPS

WAYMO

Level 4

driverless paid rides

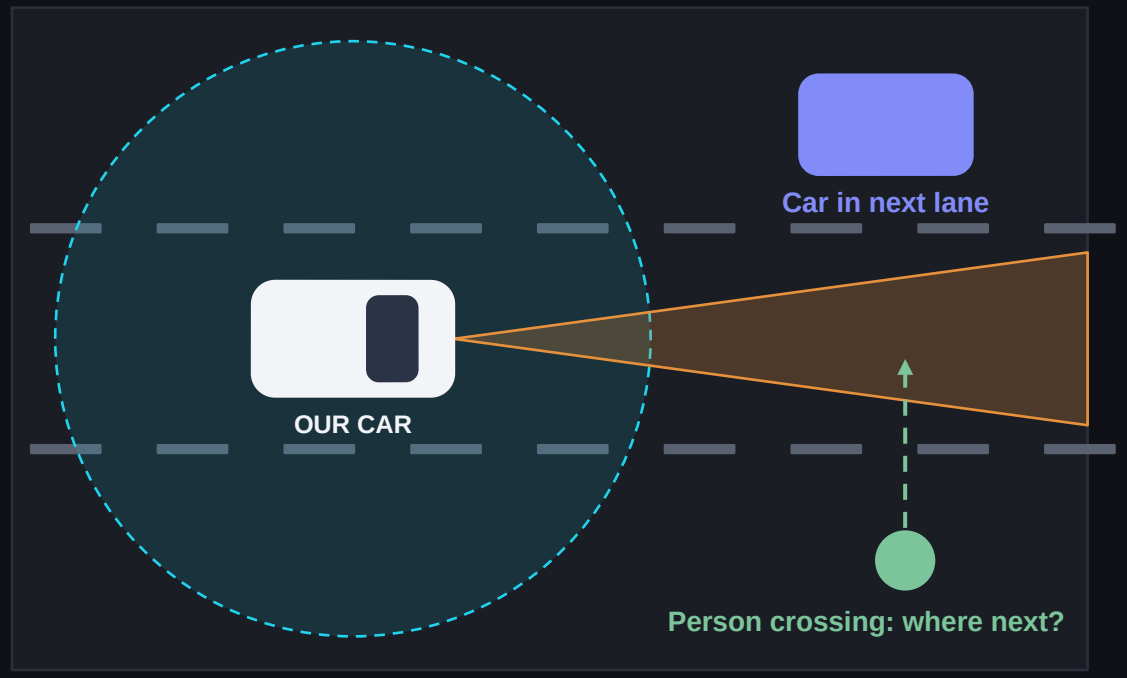
No one in the driver's seat, but only inside mapped service areas in several U.S. cities.

Perception + planning

Source: Waymo

How a self-driving car works: one loop, many times a second

Driver assistance and driverless cars run the same loop. Each step is a different kind of AI, and several are ideas students have already seen.



Cameras and lidar: 360° view of shapes, lanes and signs

Radar: distance and speed ahead, works in rain

1 Sense

Cameras, radar, lidar, GPS and a detailed map of the road.

2 Perceive

Find and label every car, cyclist, person and sign (the Module 3 idea).

3 Predict

Where will each of them be in the next few seconds?

4 Plan

Choose a safe path and speed: a repeated decision, like tonight's cases.

5 Control

Steer, brake, accelerate. Then the loop starts again.

Who is driving? The six SAE automation levels

The useful question at every level: who is responsible for driving right now? Bars show how much of the driving the car does.



L0

No automation

Warns or briefly helps; you drive.

Blind-spot warning, emergency braking

L1

Driver assistance

Helps with speed or steering.

Adaptive cruise control

L2

Partial automation

Steers and controls speed; you supervise every second.

GM Super Cruise, Ford BlueCruise, Tesla FSD (Supervised)

L3

Conditional

Drives in set conditions; you take over when asked.

Mercedes DRIVE PILOT: set highways, daylight, ≤ 40 mph (CA) Tesla FSD (Supervised)

L4

High automation

Drives itself in its service area; no driver needed there.

Waymo robotaxis

L5

Full automation

Drives anywhere a human could.

Does not exist today

Levels 0–2 · You are driving (driver support)

Levels 3–5 · The system drives while engaged

Source: SAE J3016

California DMV: DRIVE PILOT approval

What is still hard, and who is watching

Driverless cars work today only inside set roads and conditions. Regulators decide where, and can take permission away.

WHAT IS STILL HARD

Rare events

A police officer waving traffic through, debris on the highway: too rare to see often in training.

Weather and conditions

Rain, fog, snow and construction confuse sensors, so each system is approved only for set roads and conditions (its ODD).

Handing control back

At Level 3 a person must take over when asked, and people are slow to re-engage.

Proving it is safer

Beating human drivers must be shown over huge mileage, with a fair comparison.

WHO IS WATCHING

2021 · NHTSA

Companies must report crashes involving Level 2 assistance and Level 3–5 driving systems.

2023 · California DMV

Approves a Level 3 system only on set highways, in daylight, up to 40 mph.

2023 · California DMV

Suspends Cruise’s driverless permits after a crash that seriously injured a pedestrian.

When the system drives, the company answers for it. Tonight’s question: who answers for a learned decision?

WHY THIS CASE IS NEW

Until now, every model learned from recorded data

MODULE	THE DATA ALREADY RECORDED	WHAT THE MODEL LEARNED
Module 2 · Fraud	Card transactions later confirmed as fraud or not	Is this transaction fraud?
Module 3 · Healthcare	Chest X-rays labeled pneumonia or normal	Does this image show pneumonia?
Module 4 · Finance	Loans that were repaid or defaulted	Will this borrower default?
Module 5 · Manufacturing	Hours of normal compressor sensor readings	Does this hour look unusual?
Module 6 · Retail	Units actually sold; what shoppers actually bought	How many next week? What next?
Module 7 · Transportation	Nothing. Nobody can record the “right move”: it depends on what happens next.	**Which move leads to the most points? It learns by trying.**

Learning by reward: you already do it on your commute

Reinforcement learning is learning by trial and error: try, see how it went, and do more of what worked.

YOUR FIRST WEEK IN A NEW CITY: THREE WAYS TO GET TO WORK

MON	TUE	WED	THU	FRI
Highway	Side streets	Side streets	Toll road	Side streets
35 min	28 min	27 min	31 min	26 min
First try	Try another road	Repeat the best	Explore: try a new one	Repeat the best

WHAT YOU REMEMBER AFTER ONE WEEK

Highway	1 try	35 min	
Side streets	3 tries	**27 min average**	BEST
Toll road	1 try	31 min	

Next Monday you take the side streets, and keep updating as you go.

A MACHINE DOES THE SAME, THOUSANDS OF TIMES



1 Try

Pick a route, a move, or a switch.



2 Reward

A number: minutes lost, or points.



3 Update

Change the score for that choice.

Repeat, usually in a simulator, where a bad choice costs nothing.

Four ideas to carry into the three cases

Every reinforcement learning system, from a game-playing program to tonight's robot, is built from these four ideas.

1

The reward

A number after each action that says how well it went. People choose it, so people decide what "good" means.

Commute: minutes lost in traffic

Case 2: what you measure

2

A score per choice

The learner keeps a score for each choice in each situation, and nudges it after every try.

Commute: side streets ≈ 27 min

Case 1: the table of scores

3

Explore a little

Mostly pick the best-known choice, but sometimes try another, in case it is better.

Commute: Thursday's toll road

Case 1: 1 move in 5 is random

4

Rewards can be late

Some choices pay off only later, so a score counts what comes next, not just now.

Commute: detour to skip a jam

Case 3: pay now, earn later

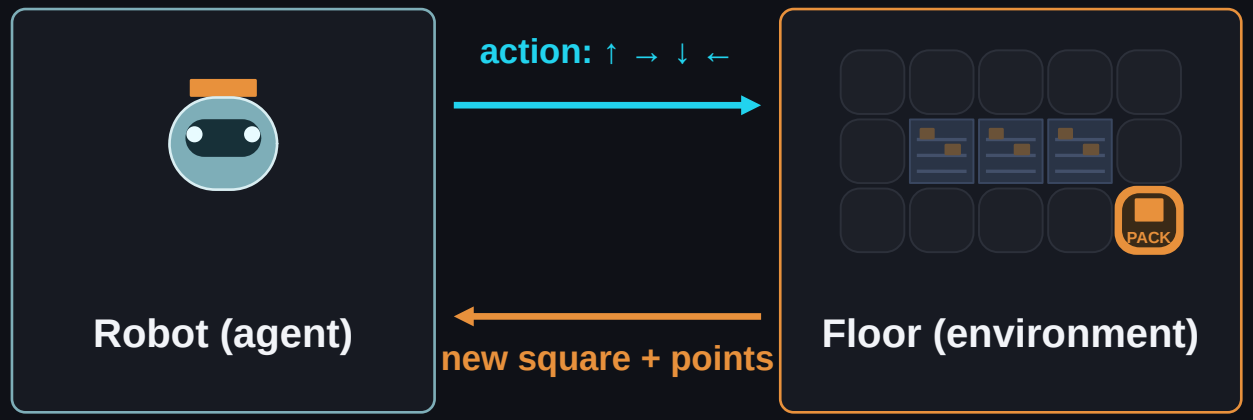
YOU HAVE MET IT BEFORE

AlphaGo beat Go champion Lee Sedol in 2016 after improving by playing against itself. ChatGPT was fine-tuned with a reward built from people's ratings of its answers.

WHAT IT NEEDS

A reward that counts what matters, lots of safe practice (usually a simulator), and rules outside the learner for what it must never do.

One loop, five words



Training = running this loop many times and writing down what each choice was worth.

The same five words describe all three cases tonight. Only what fills them changes.

State	Where the robot is: its square, such as A3.
Action	What it can do: up, right, down or left.
Reward	Points after each move: −1, or +20 at the station.
Episode	One trip, from the start square to the station.
Policy	The rule for picking a move: take the highest score.

The job: bring the shelf to the packing station

Order arrives
A customer buys an item stored on one shelf.

→

Robot drives to the shelf
It slides underneath and lifts the whole shelf.

→

Carries it to packing
Our robot learns this step.

→

Person picks the item
A human takes the item and packs it.

→

Shelf goes back
The robot returns the shelf to storage.

The real reference

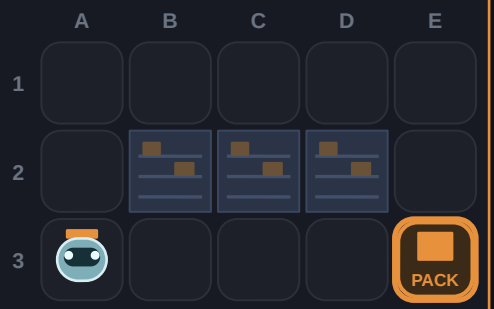
Amazon's Hercules drive units bring shelves ("pods") of items to employees who pick customer orders. Amazon says the robots read a grid of markers on the floor to find their position, and take overall direction from centralized planning software.

Amazon: robots in its fulfillment centers (updated June 2026)

Our classroom robot

A small invented floor. No map, no route: only points after each move.

Real fleets plan routes from a map. We use this floor because every piece of reinforcement learning is visible on it.

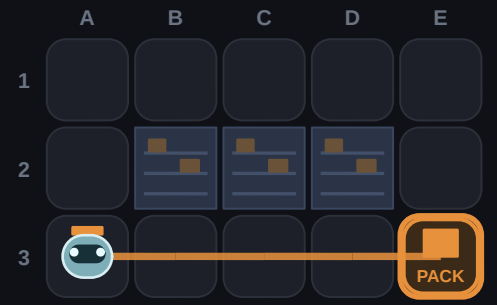


The points are the only teacher

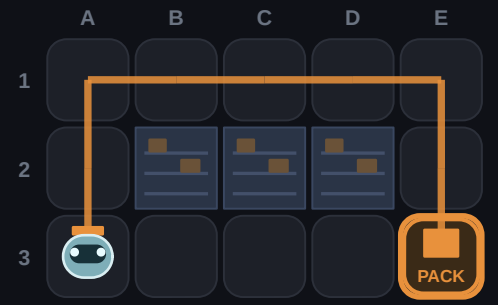
Every move	−1
Bump into a shelf or wall (it stays put)	−1
Reach the packing station	+20

So a trip earns 20 − (number of moves).

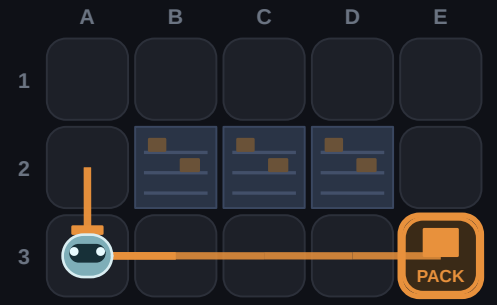
The robot is never told the route. It only sees these points.



Along the bottom: 4 moves → 16 points



Over the top: 8 moves → 12 points



Its real first practice trip: 12 moves → 8 points

That first trip, move by move

↑* ↓ ↓× → ↓× ↑× ←
→* → → ↑× →

× bump (stayed put) * random move

4 bumps, 2 random moves. It still arrived.

[Open Step 1 in the lesson: drive the robot yourself]

The model is a table of scores: four per square



Each small box is one score. Blank (dashed) = never tried. C3 shows an example.

What a score means

The points the robot expects to collect from this square to the end of the trip if it takes this arrow now. On C3: right = 18.

How it picks a move (its policy)

1. Read the four scores for its square.
2. Take the highest. A blank counts as 0. Ties: flip a coin.
3. But 1 move in 5 is a random arrow instead, so it keeps trying new things. This is exploration.

Its name: a Q-table

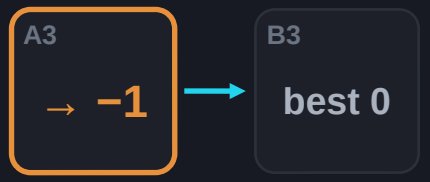
11 squares × 4 arrows = 44 numbers. That table is the entire model.

Open Step 2 in the lesson

After every move, the robot updates one score

new score = points from this move + best score where it landed

Trip 1, move 4: A3 → to B3



−1 + 0 = −1

Nothing good is known on B3 yet, so the move is only worth its cost.

Trip 1, move 12: D3 → into the station



(+20 − 1) + 0 = 19

It delivered. The trip is over, so the best score at the station is 0.

Trip 2: C3 → onto D3



−1 + 19 = 18

D3 already holds 19, so C3 becomes good too, one point less.

Only the arrow it just used changes. Everything else in the table stays the same.

Open Step 3 in the lesson: one move at a time

Trip after trip, the good news travels backward

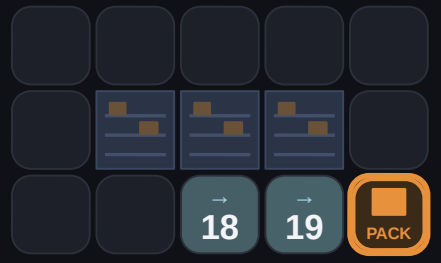
After trip 1



12 moves · 8 points

New: D3 → 19

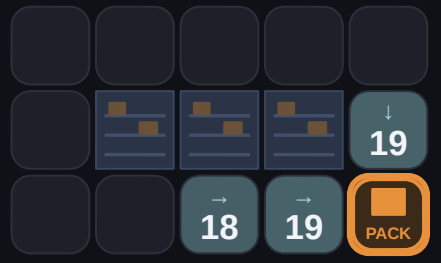
After trip 2



47 moves · −27 points

New: C3 → 18

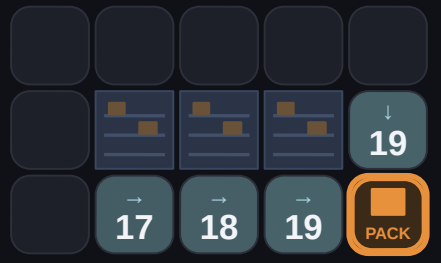
After trip 3



32 moves · −12 points

New: E2 ↓ 19

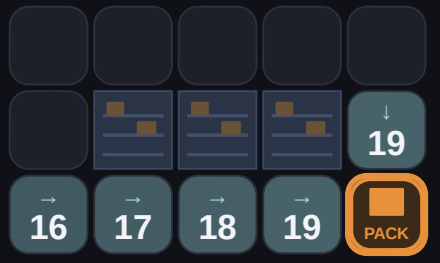
After trip 4



4 moves · 16 points

New: B3 → 17

After trip 5



6 moves · 14 points

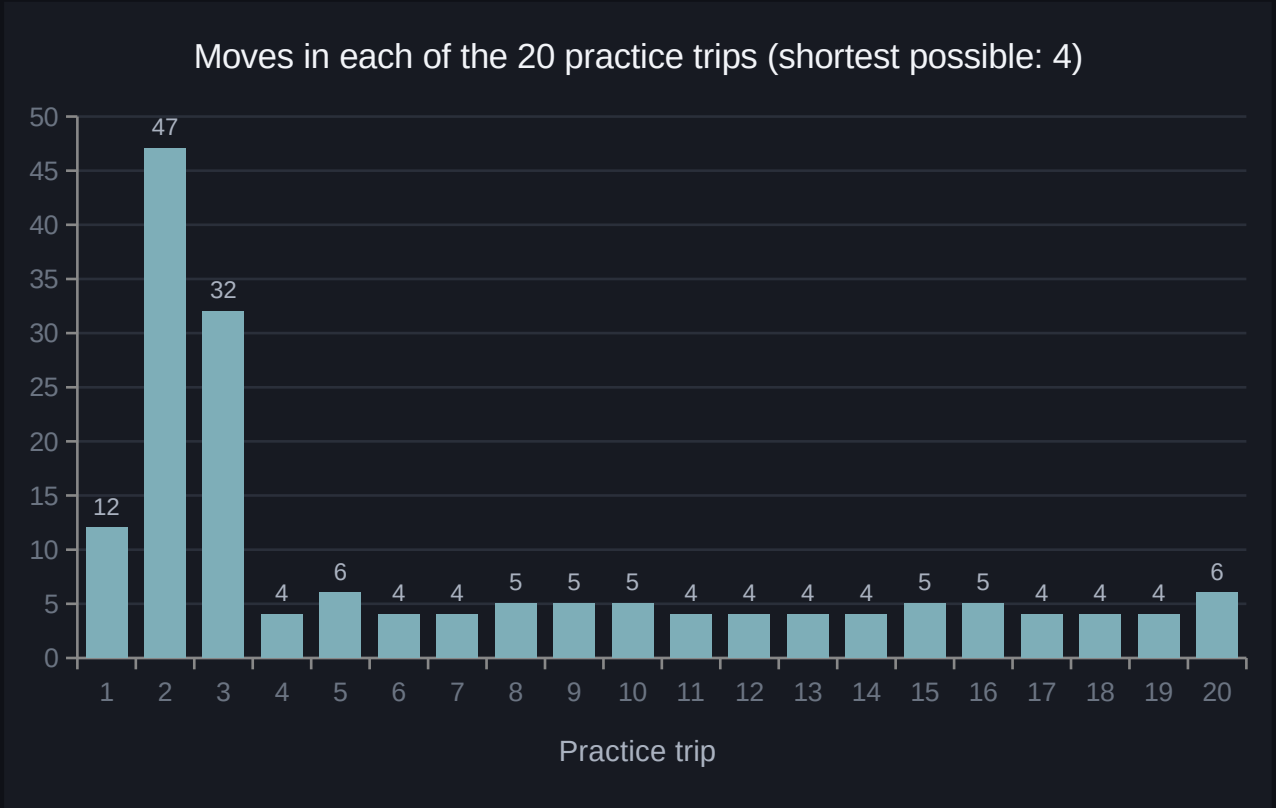
New: A3 → 16

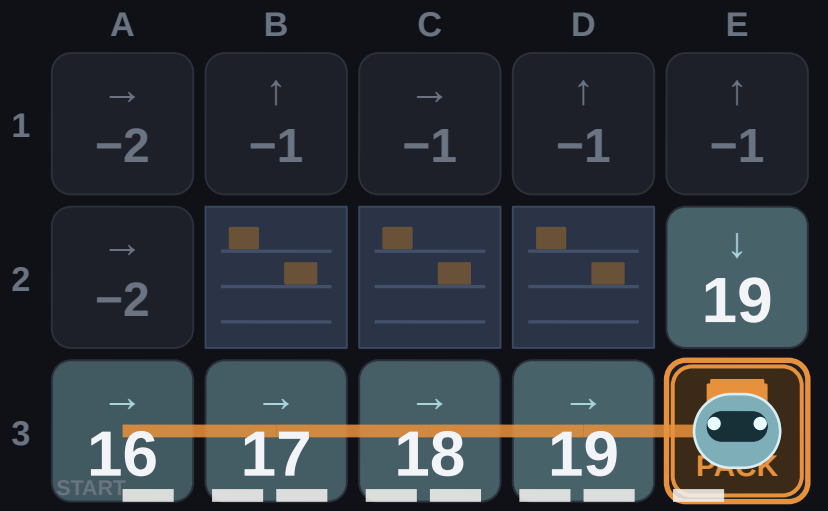
After trip 5, A3 scores 16

16 = 20 − 4: exactly the points of the shortest possible trip. Nobody told the robot the route. Each score is “points now + best score next door”, and together they add up to the best path.

Open Step 4 in the lesson: run trips 2 to 5

Test drive: always take the highest score





Orange: the robot's test drive. Dashed: shortest route found by searching the map.

Highest scores: 16 → 17 → 18 → 19

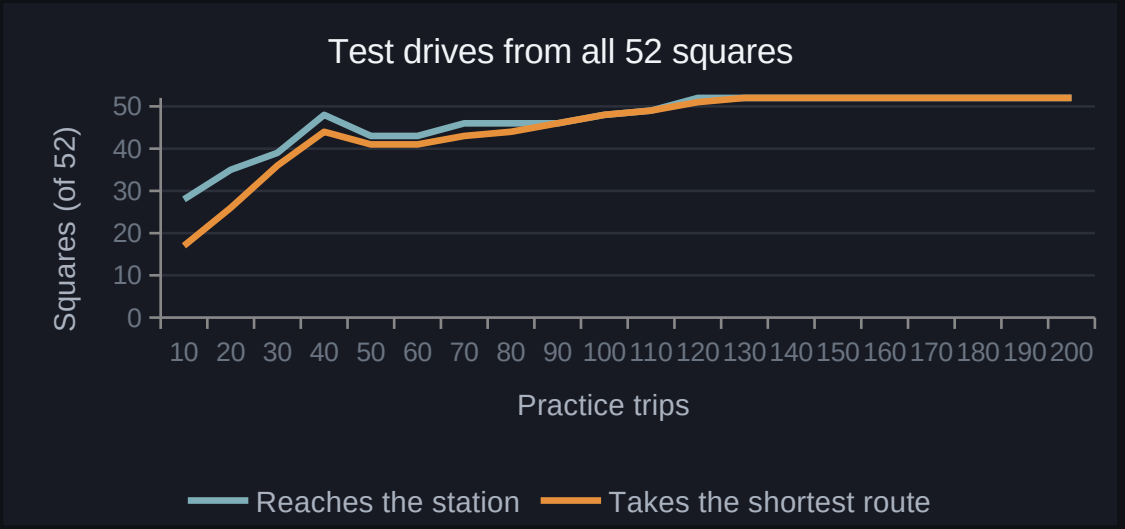
4 moves = the shortest route (4). No random moves, no learning during the test.

[Open Step 5 in the lesson: test drive from any square]

Same recipe, bigger floor: 52 squares need more practice

	A	B	C	D	E	F	G	H	I	J	K
1	↓ 7	→ 8	→ 9	→ 10	→ 11	→ 12	↓ 13	→ 14	→ 15	→ 16	↓ 17
2	↓ 8			↓ 11			↓ 14			↓ 17	↓ 18
3	↓ 9			↓ 12			↓ 15			→ 18	↓ 19
4	→ 10	→ 11	→ 12	→ 13	→ 14	→ 15	→ 16	→ 17	→ 18	→ 19	PACK
5	↑ 9			↑ 12			↑ 15			↑ 18	↑ 19
6	↑ 8			↑ 11			↑ 14			↑ 17	↑ 18
7	↑ 7	→ 8	→ 9	↑ 10	→ 11	→ 12	→ 13	→ 14	→ 15	↑ 16	↑ 17

The robot's table after 200 practice trips



Every 10 practice trips, the robot test drives from each of the 52 squares: highest score only, no random moves, no learning. Teal counts squares that reach PACK. Orange counts squares that get there in the fewest possible moves, checked against a search of the map.

32.5 → 8.4

average moves per trip, first 50 trips vs the next 50

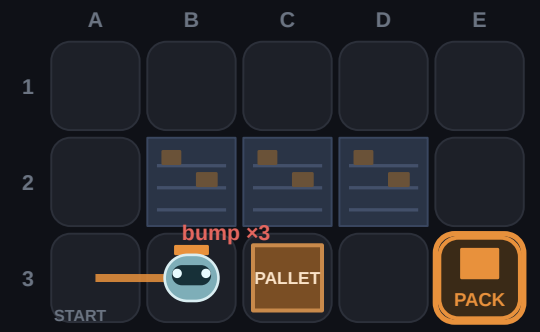
130 trips

until all 52 squares take a shortest route

[Open Step 6 in the lesson: watch the scores spread]

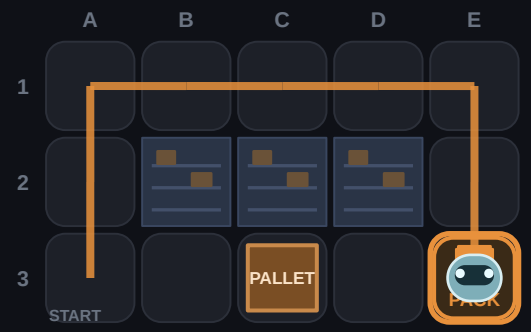
It only knows the floor it practiced on

1. A pallet appears



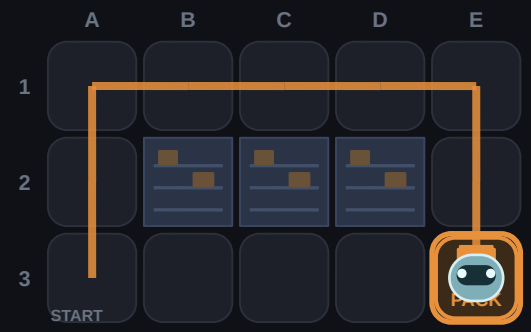
Test drive: stuck at B3. Its table still says → is worth 17.

2. Practice 10 more trips



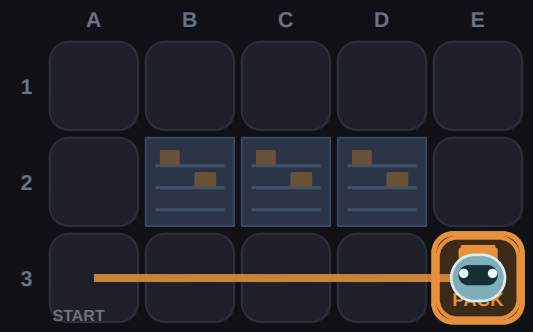
Bumps lower the blocked score. Test: 8 moves over the top.

3. Pallet removed



Still 8 moves. At A3: ↑ 12, → −7. It remembers the block.

4. Fresh table, 10 trips



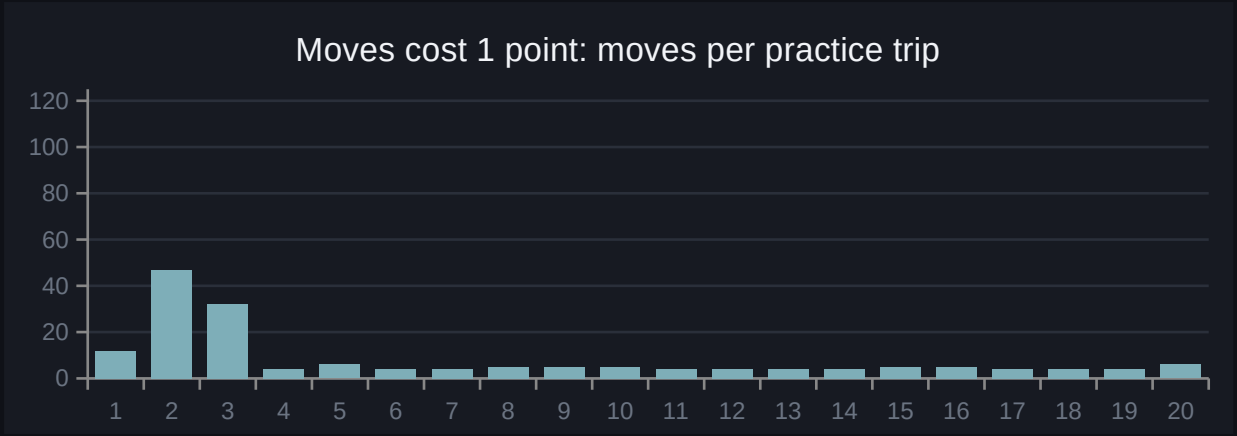
4 moves again. Starting over was faster.

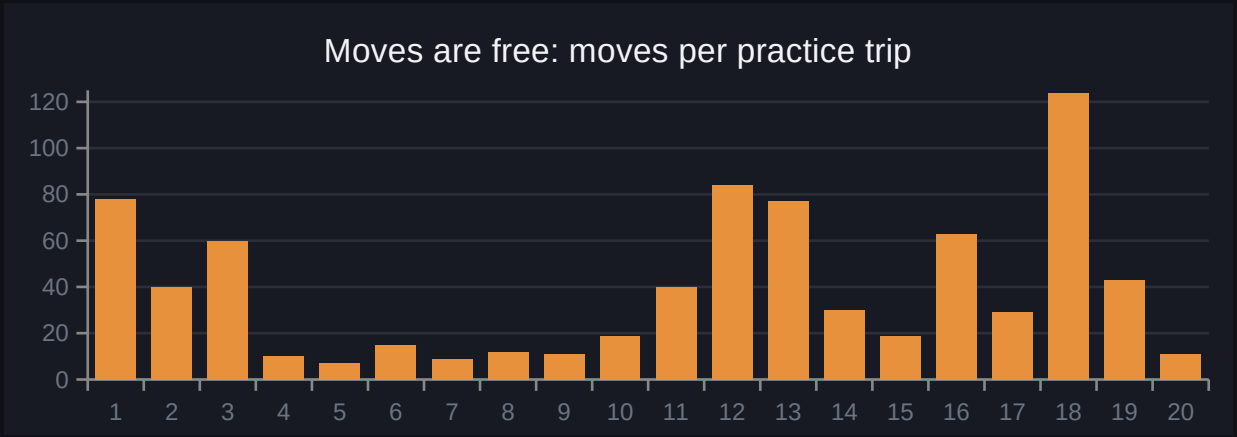
A trained model only knows the world it practiced in.

When the world changes it must practice again, and sometimes start fresh. Real warehouse systems watch the floor and re-plan routes from a live map.

Open Step 7 in the lesson: drop the pallet yourself

You get what you reward





With free moves, nothing says hurry

Every arrow at A3 scores 20. A 40-move trip is worth the same as a 4-move trip, and even a bump scores 20. In the test drive it never arrives. It did exactly what the points asked.

Next case: traffic signals

The points will be waiting time. If we only count cars, what happens to people crossing on foot, to an ambulance, or to a quiet side street that always has fewer cars? Choosing the reward is a human decision.

Open Step 8 in the lesson: train with free moves

Learn by reward, or search a map?

	Learn by reward	Search a map
Needs	Points after each move, and lots of practice	A correct map and a goal
Gets the answer by	Trying moves and updating a table of scores	Computing it directly: Dijkstra's method, 1959
Use it when	Nobody can write the answer down: traffic-light timing (next case), ride dispatch (later case)	The map is known and reliable: road routes, warehouse fleet planning
On our floor	 20 practice trips, then 4 moves	 One search, no practice: 4 moves

[Survey: traffic signal control methods (Wei et al., arXiv 1904.08117)]

[Dijkstra (1959): A note on two problems in connexion with graphs]

[Further reading: Sutton & Barto, Reinforcement Learning: An Introduction (free online)]

How traffic lights are run today

Fixed-time

Engineers set each green's length, often one plan per time of day. Neighbouring lights are timed so cars meet a "green wave". Ignores the cars actually waiting.

Actuated

Detectors (wire loops in the road, cameras, radar) sense waiting cars. The controller extends or ends a green, within a minimum and a maximum.

Adaptive

Software such as SCOOT (UK) and SCATS (Sydney) adjusts timings across many intersections from live detector counts.

Learning by reward

Tries Keep or Switch, gets points for less waiting, updates scores. Mostly research and pilot projects so far. This is what we build.

What every approach shares

Engineers write the safety rules: yellow and all-red between greens, minimum and maximum green, pedestrian crossings, emergency vehicles. The learner in this case only chooses inside them.

Open Step 1: be the traffic light yourself

The job: keep traffic moving at one intersection



The lesson’s intersection. Each press of Keep or Switch is 5 seconds; cars arrive at random.

State	Which road is green + cars waiting on each road (0, 1–2, 3–5, 6+)
Action	Keep the green, or Switch
Reward	−1 for every waiting car, every 5 seconds

Rules it cannot break (written by engineers)

- Every change: yellow, then all-red. Nobody moves for 5 s.
- A green lasts at least 10 seconds.
- A green lasts at most 60 seconds if a car waits on red.

Open Step 1: be the traffic light yourself

No finish line: nudge each score a little

When Main St is green

	0	1–2	3–5	6+
0	K	S	S	—
1–2	K	K	S	—
3–5	K	K	S	—
6+	K	K	S	—

When Side St is green

	0	1–2	3–5	6+
0	S	K	S	—
1–2	S	K	S	—
3–5	S	S	S	—
6+	S	S	S	—

Rows: cars waiting on Main St. Columns: on Side St. K = keep, S = switch, — = never seen.

64 scores: 32 situations × 2 choices. Blank scores count as 0, and every real outcome is negative, so untried choices look attractive early on.

new score = old + **0.1** × (points now + **0.9** × best score next − old)

0.1: nudge, don’t jump

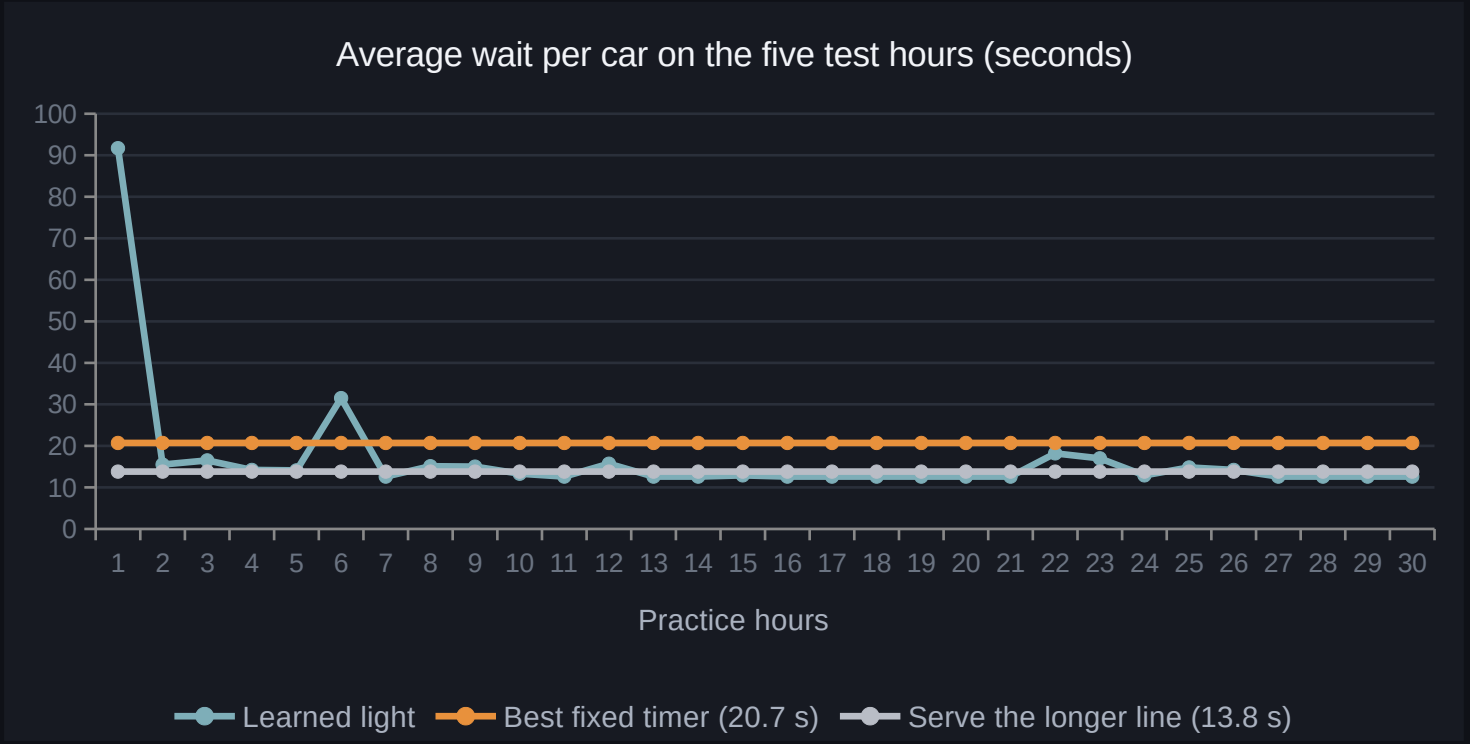
Traffic is random. One unlucky tick should not rewrite the score, so each tick moves it only a tenth of the way.

0.9: later counts less

No finish line means points never stop. Multiplying by 0.9 keeps scores finite and values soon over later.

Open Steps 2–3: the grid, then one tick at a time

30 practice hours turn the grid into a rule



How we test: after every practice hour, freeze the table and run the light on the same 5 hours of test traffic (highest score only, no random tries, no learning). Teal is the learned light. The two flat lines are simple lights run on the same cars, for comparison (next slide).

91.7 s

after hour 1: blank table

12.6 s

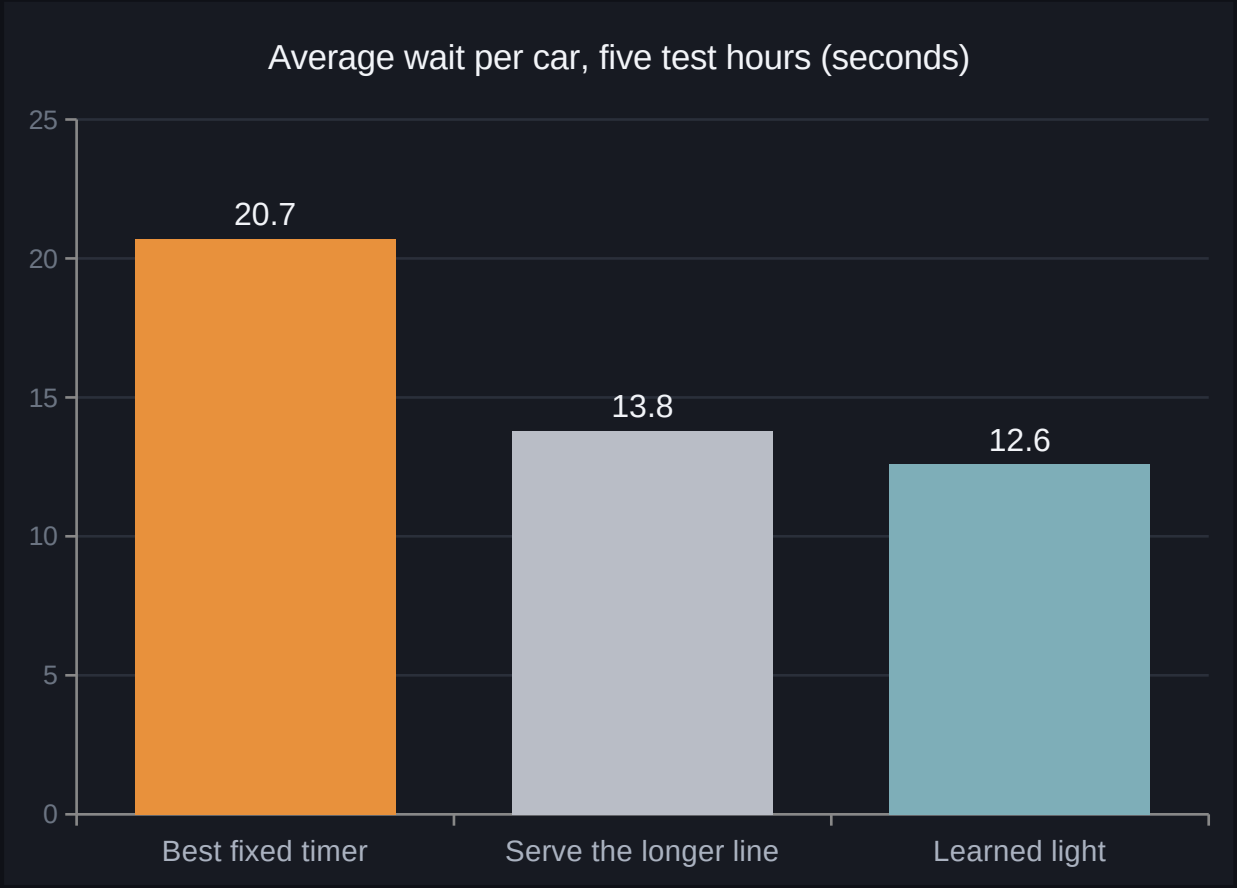
after 30 hours

Test, don’t trust

the line still jumps: always re-test before use

[Open Step 4: practice hour by hour]

Same cars, three lights: is learning worth it?



LIGHT	AVG	LONGEST	CARS
Fixed timer 35/20 s	20.7 s	72 s	406
Longer line	13.8 s	51 s	407
Learned (30 h)	**12.6 s**	**47 s**	**407**

THE THREE LIGHTS · same 5 test hours, same safety rules

Fixed timer: Main 35 s, Side 20 s, repeat (best split found)
Longer line: switch when the red road's line is longer
Learned: the 30-hour table, always the higher score

Honest conclusion

Learning roughly halves waiting compared with a fixed timer (20.7 s → 12.6 s), but a three-line rule gets most of the way (13.8 s). Always ask: better than what?

[Open Step 5: watch all three lights on the same cars]

You get what you measure

The experiment: suppose sensors exist only on Main St, so the points count only Main St cars. Same light, same 30 practice hours; then we test it counting every car.

THE POINTS COUNT	AVG WAIT	LONGEST SIDE ST WAIT
Every waiting car	12.6 s	47 s
Only Main St cars	41.6 s	4.1 min
Only Main St cars, no timing rules	**506.3 s**	**59.2 min**

Timing rules (slide 24): a green lasts at least 10 s, and at most 60 s when a car waits on red.



Main St only: Side St cars pile up

Rules limit the damage. The fix is in the points.

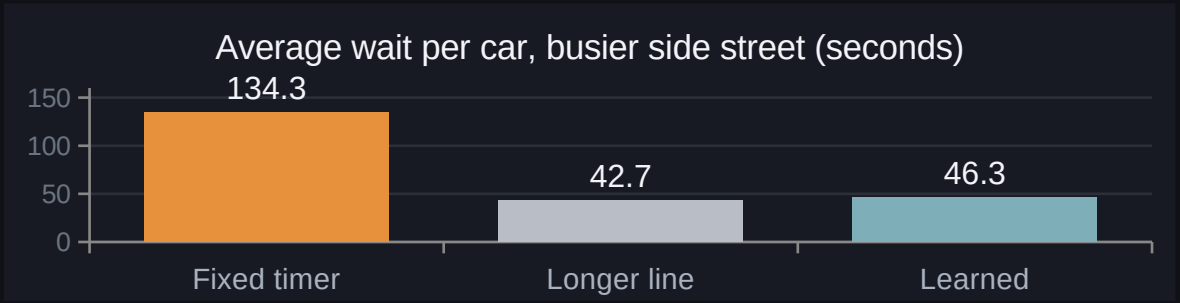
Whatever the reward does not count is invisible to the learner: side streets, buses, cyclists, people walking. Choosing the reward is choosing whose time matters.

Open Step 6: train with different points

Before a city switches it on

The experiment: a school opens and Side St gets twice as busy. Nobody retrains anything; the three lights from slide 27 face traffic they never practiced on.

SIDE ST TWICE AS BUSY	AVG WAIT	LONGEST
Fixed timer (same timings)	134.3 s	9.2 min
Serve the longer line	42.7 s	2.5 min
Learned, not retrained	**46.3 s**	**2.8 min**



Open Step 7 and the Signal lab

A checklist before switching it on

- Safety rules stay outside the learner: yellow, all-red, minimum and maximum green, pedestrians, emergency vehicles.
- The points count everyone: every road, buses, bikes, people walking.
- A fair test against a fixed timer and a simple rule, on real recorded traffic.
- Monitor after launch; retrain or fall back when traffic changes.
- A named office answers for its decisions.

How ride-hail cars find riders today

1 · A rider requests

From an app, with pickup and drop-off.

2 · The app matches

Offers the trip to a nearby free driver, within seconds.

3 · The ride

The driver picks up, drives, and is paid for the trip.

4 · Between rides

The driver decides where to wait. Driving empty = deadheading.

How apps steer drivers

- Demand heat maps: where requests are coming from now
- Surge prices when requests outnumber drivers
- Bonuses for trips in certain areas or times

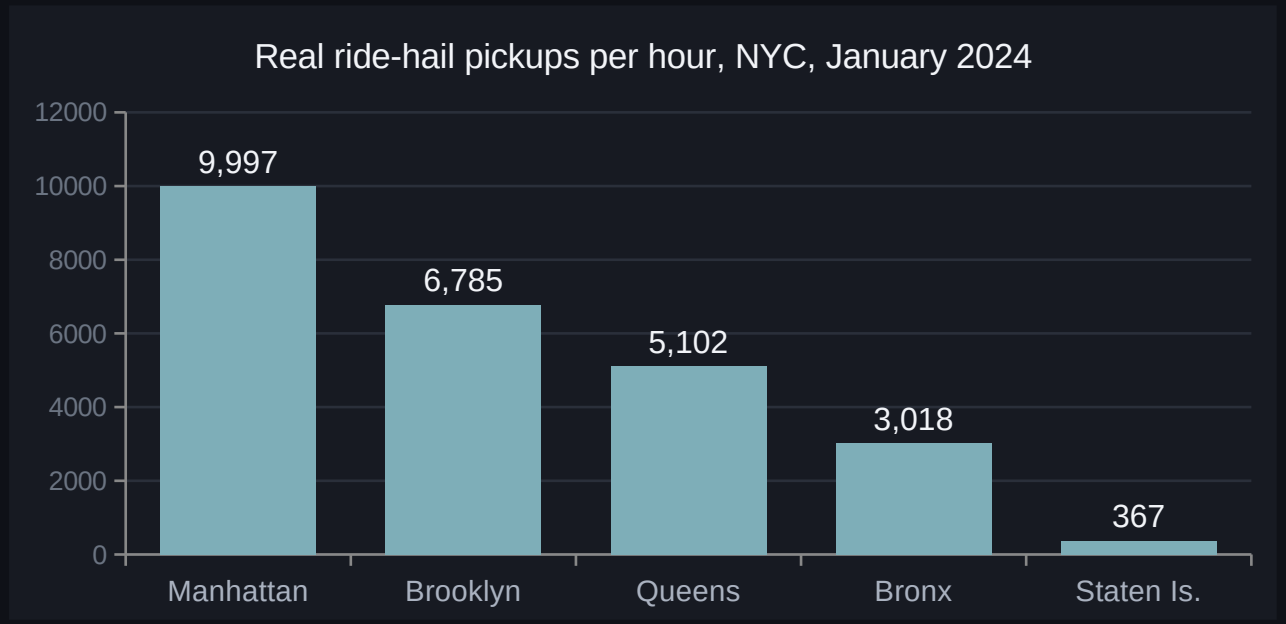
Where learning by reward comes in

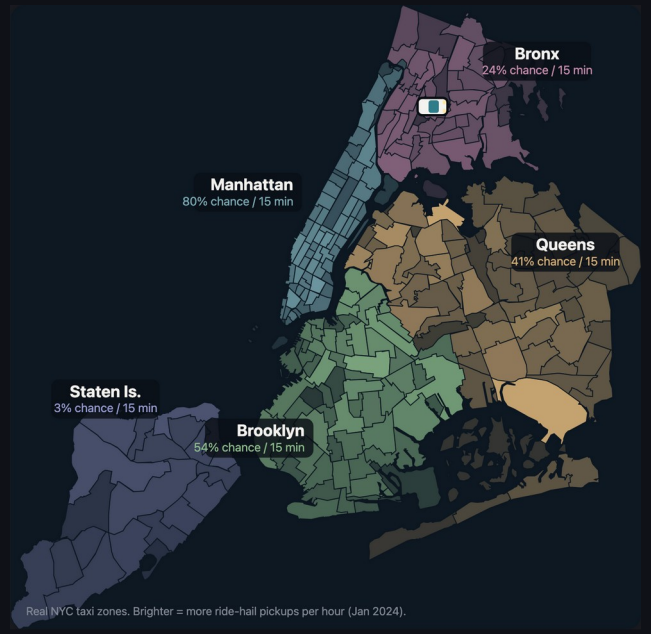
Platform researchers have published reinforcement learning for matching and for moving idle cars (e.g. DiDi, KDD 2018). Tonight: one driver, one question, where should I wait? On real New York trips.

Open Step 1: drive a shift yourself

The job: where should an empty ride-hail car wait?

The case: one ride-hail driver, one 8-hour shift, real New York trips. Every 15 minutes without a rider, the driver decides: wait in this borough, or drive empty to another. The bars and map show where real requests start.





State

Borough the driver is in

Action

Wait here, or drive empty to another

Reward

Dollars: ride pay in, empty miles out

Open Step 1: drive a shift yourself

19.7 million real trips, two named assumptions

Stages 1 and 2: each row sums up one borough from the real trips, and the driver's simulation uses these numbers. Two things the data cannot tell us had to be assumed (right).

BOROUGH	PICKUPS	TYPICAL PAY	RIDE TIME	CHANCE*
Manhattan	39.6%	$17.11	16 min	80%
Brooklyn	26.9%	$15.90	15 min	54%
Queens	20.2%	$19.11	17 min	41%
Bronx	11.9%	$14.79	14 min	24%
Staten Island	1.5%	$15.81	13 min	3%

* Chance of a request while waiting 15 minutes: our assumption, not in the data.

Source: NYC Taxi & Limousine Commission, High Volume For-Hire Vehicle trip records, January 2024. 19,663,930 trips; only borough totals and averages are kept.

[NYC TLC trip record data]

[Open Step 2: the data]

What the data gives

- Rides starting per hour
- Where rides go
- Real driver pay and trip time

What we assumed

- How often a waiting driver gets a request
- Empty driving: 50¢ a mile

Delayed reward: money later counts a little less

The question: why would a driver ever pay to drive empty? Each choice's score estimates the dollars it leads to over the rest of the shift, with later money counting a little less.

new score = old + 0.05 × (**dollars now** + **0.9^blocks × best score where it ends up** − old)

Now

Drive empty Bronx → Manhattan: −$2.79

→

15 minutes later

In Manhattan: an 80% chance of a request every 15 minutes

→

The shift goes on

More rides per hour than waiting in the Bronx (24%)

After training, the Bronx scores

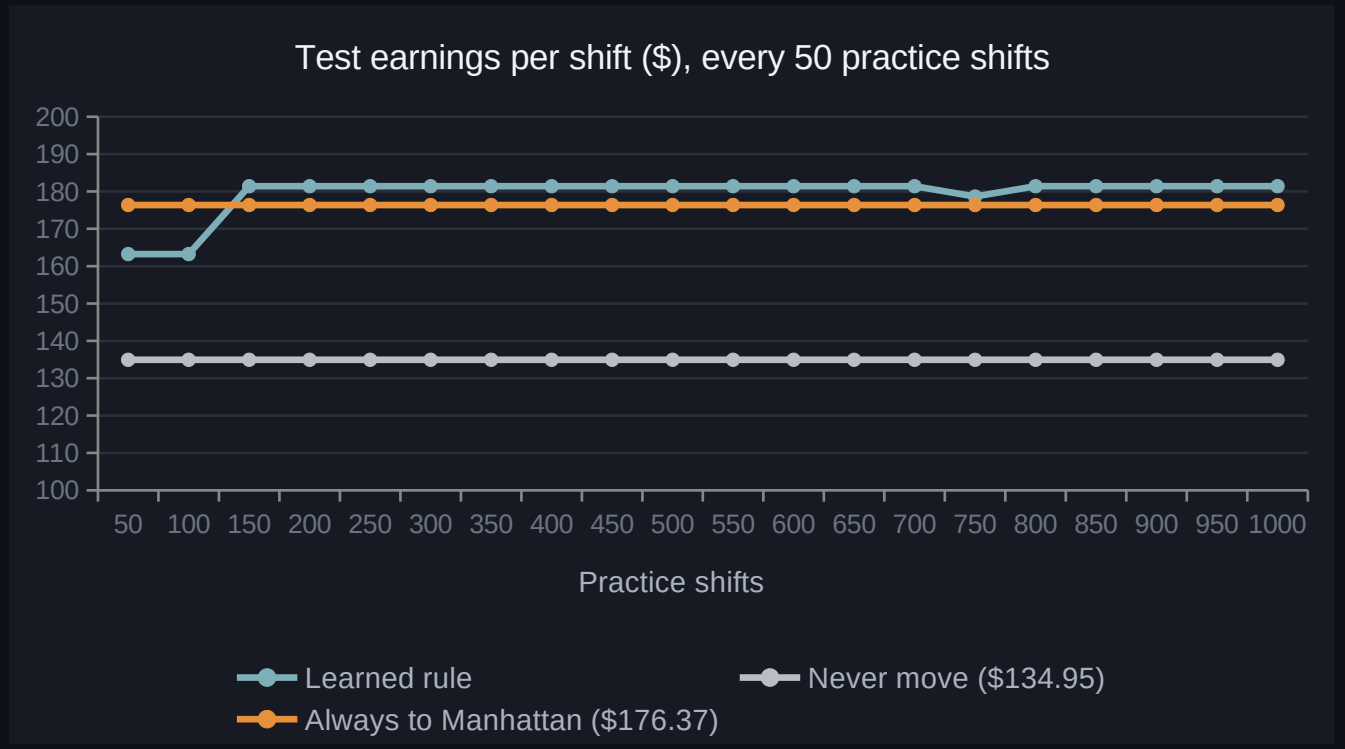
Wait here: 51.4 Drive to Manhattan: 57.8
Each score = dollars expected from here on.
The empty drive wins because of what it leads to.

Why 0.9 per 15 minutes?

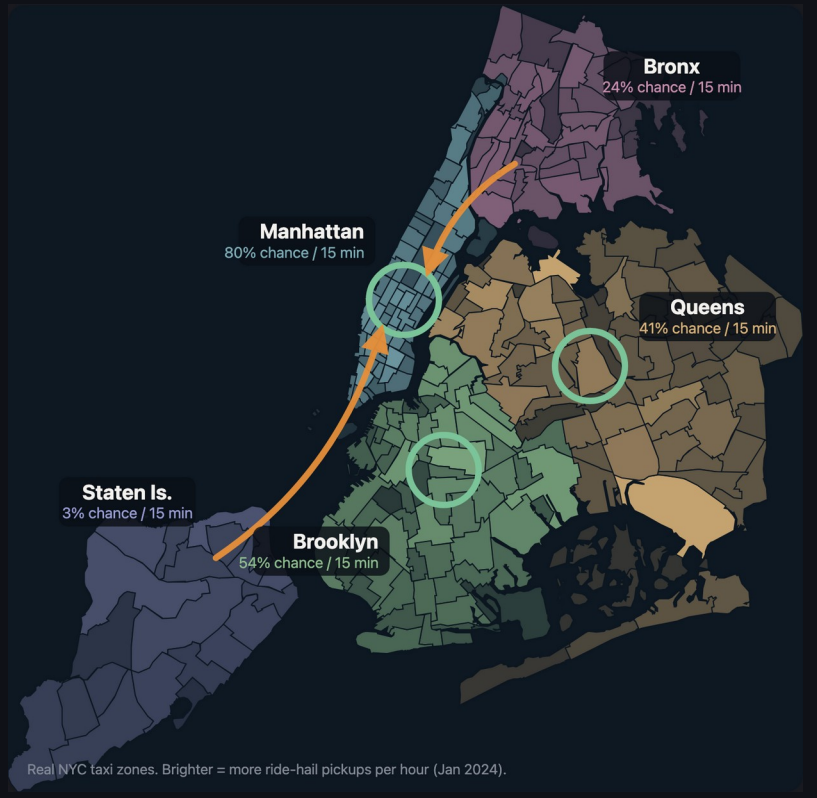
$1 in 15 minutes is worth 90¢ now; in an hour about 66¢. With 0, only the next 15 minutes would matter.

Open Step 3: one decision at a time

A thousand practice shifts become a rule



How we test: practice shifts start in a random borough and 1 choice in 5 is random. Every 50 shifts we freeze the table and run it on the same 2,000 test shifts (no random choices, no learning). The flat lines are two simple rules on the same shifts.

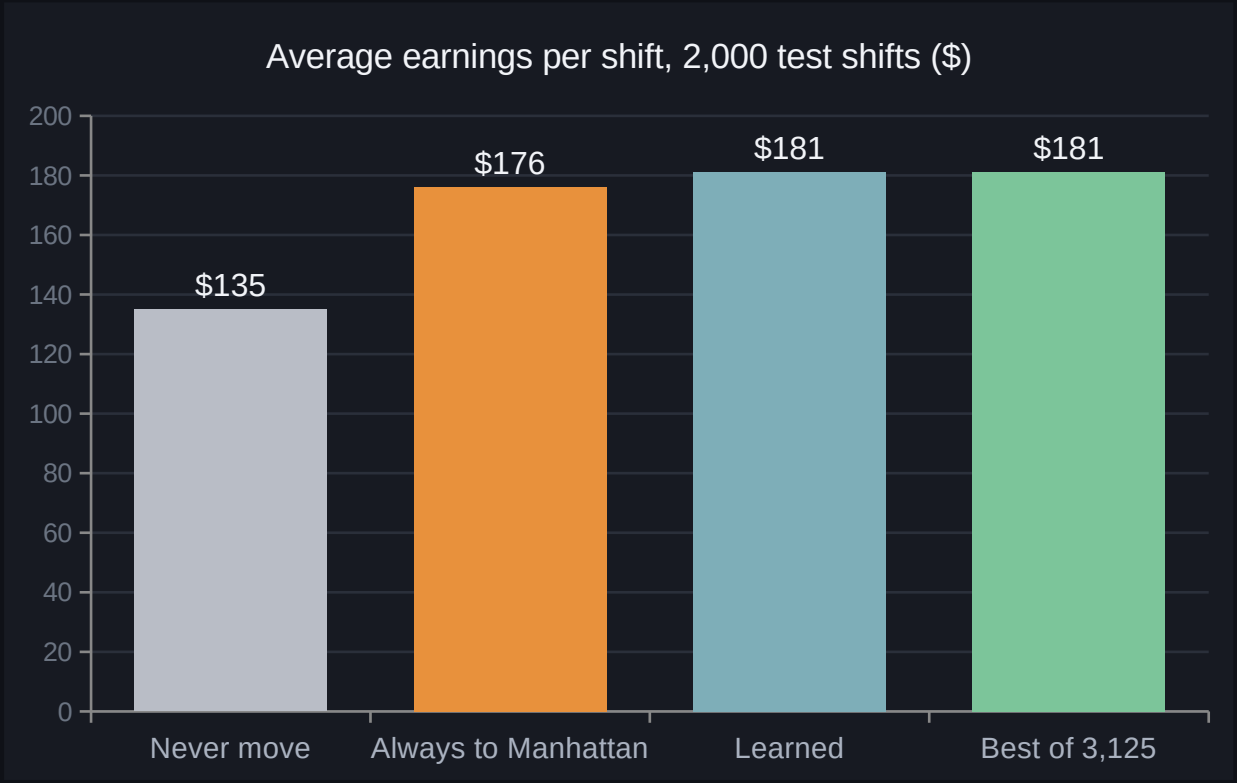


The learned rule: **Green ring** = wait there. **Orange arrow** = drive empty there.

[Open Step 4: practise 1,000 shifts]

Validation: as good as checking all 3,125 rules

The experiment: with 5 boroughs and 5 choices there are 3,125 possible where-to-wait rules. We test every one on the same 2,000 shifts and compare. Right: retrain with a different look-ahead (0 = only the next 15 minutes count).



LOOK-AHEAD	$ / SHIFT	DRIVES EMPTY
0	$134.95	never
0.5	$151.34	Bronx → Manhattan
0.9	**$181.40**	**Bronx → Manhattan, Staten Is. → Manhattan**

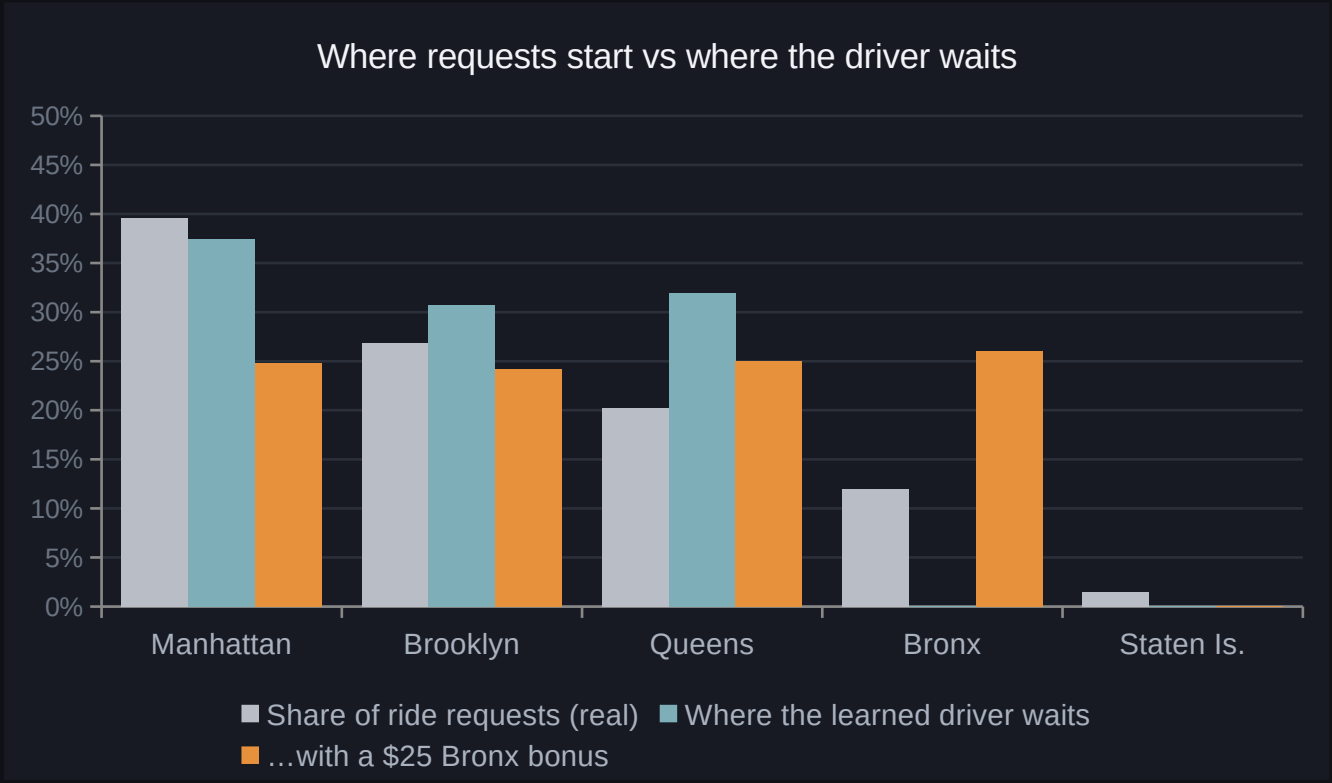
Why this matters

Only a driver that values future rides ever pays to drive empty. Choosing how far ahead the learner looks is a design decision, like choosing its points.

Open Steps 5–6: test it, then change the look-ahead

Good for the driver is not good for every rider

The question: if every driver followed this rule, which riders would be left without a car? The bars compare where requests start (real) with where the learned driver waits. The table shows real rider waits (Uber trips).



RIDERS WAIT (REAL)	MEDIAN	AVERAGE
Manhattan	2.15 min	2.74 min
Brooklyn	2.72 min	3.24 min
Queens	2.95 min	3.59 min
Bronx	2.85 min	3.37 min
Staten Island	**3.78 min**	**4.32 min**

With a $25 Bronx pickup bonus

Bronx gets 26% of waiting time. The platform pays $30.98 per shift. Staten Island still gets none.

Open Step 7: who is served

Picture it: 30 cars follow one rule

The experiment: 30 cars all follow one rule for an 8-hour shift on New York's real taxi zones, with the same riders in both pictures. A request that no waiting car takes within 30 minutes is missed.



Rule learned from driver earnings



Rule learned with a $25 Bronx bonus

BOROUGH	RIDERS	LEARNED	BONUS
Manhattan	116	116	116
Brooklyn	62	59	49
Queens	63	52	63
Bronx	**39**	**0**	**37**
Staten Is.	8	0	0

Riders = requests in the shift. Learned, Bonus = requests a waiting car took.

A picture, not a measurement

All 30 cars obey one rule. No competition, riders do not react. It shows the trade-off; it does not size it.

Open Step 7: watch 30 cars follow each rule

Same recipe, three jobs

	Case 1 · Warehouse robot	Case 2 · Traffic light	Case 3 · Ride-hail driver
State	The square it is on	Green road + cars waiting on each	The borough it is in
Action	↑ → ↓ ←	Keep or Switch	Wait, or drive to another borough
Reward	−1 per move, +20 at the station	−1 per waiting car, every 5 s	Real ride pay, minus empty miles
Model	44 scores (small floor)	64 scores	25 scores
New idea	The loop; test with randomness off	No finish line; rules outside the learner	Delayed reward; real data
Honest result	Search on a known map finds the same route	A 3-line rule is almost as good (13.8 s vs 12.6 s)	Best for drivers leaves two boroughs behind

Safety and accountability

1. What can it never do?

Safety lives in rules outside the learner: yellow and all-red, minimum and maximum green, emergency stops.

2. Who is counted?

The reward decides whose time matters: side streets, buses, people walking, riders in every borough.

3. Better than what?

Compare with a timer, a simple rule or a map search, on the same data, before and after launch.

4. Who answers for it?

A named owner monitors it, retrains or falls back, and reports problems.

Real example: in the US, NHTSA's Standing General Order (2021) requires companies to report crashes involving automated driving systems and Level 2 driver-assistance systems. Sensing that makes smart transport work is also data about people: plan for privacy from the start.

Check yourself

1. What does a learner know, and what does it receive after each action?
2. Why does every test turn off the random tries?
3. Why does the traffic light need the 0.1 and the 0.9?
4. What happened when the points counted only Main St cars?
5. Why would a driver ever pay to drive empty?
6. Who is left out by a reward that only counts driver earnings?

MATERIALS

Your turn: the labs and notebooks

Case 1 · Warehouse robot

Lesson + lab

M7_Robot_Training_Lab.html

Notebook (read-only copy)

M7_Notebook_Warehouse_Robot.html

Case 2 · Traffic light

Lesson + lab

M7_Traffic_Signal_Lab.html

Notebook (read-only copy)

M7_Notebook_Traffic_Signal.html

Case 3 · Ride-hail driver

Lesson + lab

M7_Ride_Hail_Lab.html

Notebook (read-only copy)

M7_Notebook_Ride_Hail_Dispatch.html

Every notebook follows the five stages:

1 Data and provenance

2 EDA and preparation

3 Model training

4 Validation and operating policy

5 Prediction and handoff

Homework idea: one lab experiment per case. Write your prediction first, then run it and explain the result.